



Факультет Систем Управления и Робототехники

ИСПОЛЬЗОВАНИЕ РЕГУЛЯТОРОВ ДЛЯ УПРАВЛЕНИЯ ДВИГАТЕЛЕМ

Аннотация – В данной работе исследованы различные типы регуляторов: релейный, пропорциональный (П), пропорционально-дифференциальный (ПД), пропорционально-интегральный (ПИ), пропорционально-интегрально-дифференциальный (ПИД) и пропорционально-дифференциально-интегральный (ПДИ). Для каждого регулятора собраны экспериментальные данные при управлении двигателем EV3 с целевым углом поворота 90° . Проведён анализ показателей качества переходного процесса: времени переходного процесса, перерегулирования, установившейся ошибки. Представлены результаты моделирования в Simulink и анализ расхождений между экспериментом и моделью.

Ключевые слова: регуляторы; ПИД; ПДИ; EV3; моделирование; MATLAB; Simulink

Выполнили

Мостафа Эхаб Ахмед Абделвахаб Мохамед¹,
Ануфриев Андрей Сергеевич²

¹503341

²465029

Проверил

Овчаров Алексей Олегович

2 июня 2026 г.

Содержание

1	Постановка задачи	3
2	Теоретические основы	4
2.1	Показатели качества переходного процесса	4
2.2	Релейный регулятор	4
2.3	Пропорциональный (П) регулятор	4
2.4	Пропорционально-дифференциальный (ПД) регулятор	4
2.5	Пропорционально-интегральный (ПИ) регулятор	5
2.6	ПИД-регулятор	5
2.7	ПДИ-регулятор	5
3	Описание установки и методика	6
4	Результаты экспериментов и моделирования	7
4.1	Релейный регулятор	7
4.1.1	Описание	7
4.1.2	Экспериментальные результаты	7
4.1.3	Моделирование в Simulink	7
4.2	Пропорциональный (П) регулятор	9
4.2.1	Исследованные коэффициенты	9
4.2.2	Экспериментальные результаты	9
4.2.3	Моделирование в Simulink	9
4.3	Пропорционально-дифференциальный (ПД) регулятор	11
4.3.1	Исследованные коэффициенты	11
4.3.2	Экспериментальные результаты	11
4.3.3	Моделирование в Simulink	11
4.4	Пропорционально-интегральный (ПИ) регулятор	13
4.4.1	Исследованные коэффициенты	13
4.4.2	Экспериментальные результаты	13
4.4.3	Моделирование в Simulink	13
4.5	ПИД-регулятор (метод Циглера-Никольса)	15
4.5.1	Нахождение коэффициентов	15
4.5.2	Экспериментальные результаты	15
4.5.3	Моделирование в Simulink	15
4.6	ПДИ-регулятор	17
4.6.1	Порядок настройки	17
4.6.2	Экспериментальные результаты	17
4.6.3	Моделирование в Simulink	17
5	Сравнительный анализ	19
5.1	Сравнение всех регуляторов	19
5.2	Таблица показателей качества	19
6	Заключение	20
7	Листинги кода	21
7.1	Relay Regulator	21
7.2	Proportional (P) Regulator	21
7.3	Proportional-Derivative (PD) Regulator	22

7.4	Proportional-Integral (PI) Regulator	22
7.5	MATLAB Script	23

1 Постановка задачи

Цель данной лабораторной работы — исследование и сравнение различных типов регуляторов при управлении двигателем постоянного тока EV3.

Объект управления — крупный двигатель EV3. Управляющий сигнал — скважность напряжения (duty cycle) в диапазоне $[-100\%, 100\%]$. Целевое значение регулируемой переменной — угол поворота вала двигателя $\theta^* = 90^\circ$.

Для достижения цели необходимо:

1. Реализовать на языке Python следующие типы регуляторов: релейный, пропорциональный (П), пропорционально-дифференциальный (ПД), пропорционально-интегральный (ПИ), ПИД и ПДИ.
2. Получить экспериментальные графики $\theta(t)$ для каждого регулятора.
3. Построить схему моделирования в Simulink и сравнить её результаты с экспериментом.
4. Проанализировать показатели качества переходного процесса и расхождения между моделью и экспериментом.
5. Настроить ПИД-регулятор методом Циглера-Никольса.

2 Теоретические основы

2.1 Показатели качества переходного процесса

Качество работы системы управления оценивается тремя основными показателями:

- **Перерегулирование** σ — максимальное отклонение переходного процесса от установившегося значения $f_{уст}$:

$$\sigma = \frac{f_{\max} - f_{уст}}{f_{уст}} \quad (1)$$

- **Время переходного процесса** $t_{\text{пн}}$ — время от начала воздействия до момента, с которого отклонение регулируемой величины не превышает 5% от установившегося значения.
- **Установившаяся ошибка** $e_{уст} = \theta^* - \theta(\infty)$ — остаточное отклонение в установившемся режиме.

2.2 Релейный регулятор

Управление задаётся формулой:

$$U = \begin{cases} 100\%, & \theta < \theta^*, \\ 0, & \theta = \theta^*, \\ -100\%, & \theta > \theta^*. \end{cases} \quad (2)$$

Регулятор прост, обладает высоким быстродействием, однако неизбежно генерирует колебания вокруг целевого значения, поскольку вблизи θ^* двигатель имеет большую скорость и не успевает остановиться точно в заданной точке.

2.3 Пропорциональный (П) регулятор

$$U = k_p(\theta^* - \theta) = k_p e(t) \quad (3)$$

Коэффициент k_p определяет, насколько сильно ошибка $e(t) = \theta^* - \theta(t)$ влияет на управляющее воздействие. При малом k_p система реагирует медленно, при большом — возникают колебания.

2.4 Пропорционально-дифференциальный (ПД) регулятор

$$U = k_p e(t) + k_d \dot{e}(t) \quad (4)$$

Дифференциальная составляющая притормаживает систему перед целевым значением и снижает перерегулирование. Производная вычисляется численно:

$$\dot{e}(t) \approx \frac{e_n - e_{n-1}}{\Delta t} \quad (5)$$

2.5 Пропорционально-интегральный (ПИ) регулятор

$$U = k_p e(t) + k_i \int_0^t e(\tau) d\tau \quad (6)$$

Интегральная составляющая накапливает ошибку и устраняет установившуюся ошибку. Интеграл вычисляется методом прямоугольников:

$$\int_0^t e(\tau) d\tau \approx \sum_{i=1}^n e_i \Delta t \quad (7)$$

2.6 ПИД-регулятор

$$U = k_p e(t) + k_i \int_0^t e(\tau) d\tau + k_d \dot{e}(t) \quad (8)$$

Объединяет все три составляющие. Настройка методом Циглера-Никольса:

$$k_p = 0.6 K, \quad k_i = \frac{2 k_p}{T}, \quad k_d = \frac{k_p T}{8} \quad (9)$$

где K — критический коэффициент П-регулятора, при котором возникают незатухающие колебания, T — период этих колебаний.

2.7 ПДИ-регулятор

ПДИ-регулятор имеет ту же формулу, что и ПИД:

$$U = k_p e(t) + k_i \int_0^t e(\tau) d\tau + k_d \dot{e}(t) \quad (10)$$

Принципиальное отличие — **порядок подбора коэффициентов**: сначала подбирается k_d (дифференциальная составляющая), затем k_i (интегральная). Это позволяет сначала обеспечить хорошую динамику, а затем устранить статическую ошибку.

3 Описание установки и методика

Экспериментальная установка: робот LEGO Mindstorms EV3, крупный двигатель EV3 со встроенным датчиком положения, компьютер с Python и MATLAB.

Параметры экспериментов приведены в таблице 1.

Параметр	Значение
Целевой угол поворота θ^*	90°
Диапазон управления U	$[-100\%, 100\%]$
Период дискретизации Δt	50 мс
Длительность эксперимента	5–10 с

Таблица 1: Параметры экспериментов

В каждой итерации цикла (период $\Delta t = 50$ мс) программа: читает текущий угол θ , вычисляет ошибку $e = \theta^* - \theta$, рассчитывает управляющий сигнал U по закону выбранного регулятора, ограничивает $U \in [-100, 100]$ и записывает данные в файл.

Схема моделирования в Simulink включает электрическую модель двигателя (параметры R , L , k_m , k_e , J) с моментом силы трения и блок регулятора. По сравнению с лабораторной работой 2 схема дополнена блоком момента трения $M_{\text{тр}}$, что позволяет точнее воспроизвести поведение реального двигателя вблизи целевого значения.

4 Результаты экспериментов и моделирования

4.1 Релейный регулятор

4.1.1 Описание

Реализованы два варианта:

- $|U| = 50\%$ — меньшая скорость, меньше колебания;
- $|U| = 100\%$ — высокое быстродействие, но значительные колебания.

4.1.2 Экспериментальные результаты

На рис. 1 приведены переходные процессы для обоих вариантов.

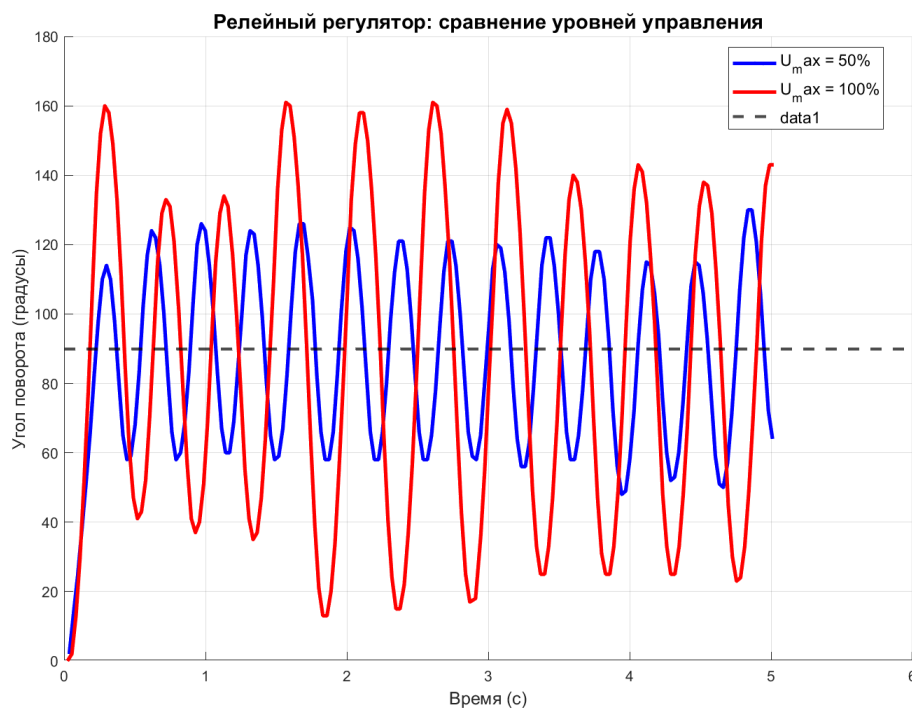


Рис. 1: Релейный регулятор: сравнение $|U| = 50\%$ и $|U| = 100\%$

При $|U| = 100\%$ система быстро достигает окрестности θ^* , однако из-за большой скорости двигатель её проскакивает и в дальнейшем колеблется около целевого значения. Уменьшение управляющего сигнала до 50% снижает амплитуду колебаний, но замедляет реакцию системы. Установившаяся ошибка неустранима: при достижении θ^* знак управления меняется, что не позволяет точно остановиться.

4.1.3 Моделирование в Simulink

Для релейного регулятора в Simulink использован блок **Relay** с параметрами: «Switch on point» = 0, «Switch off point» = 0, «Output when on» = $-U_{\max}$, «Output when off» = $+U_{\max}$. На вход блока подаётся ошибка $e(t)$: при $e > 0$ (двигатель не дошёл) выдаётся $+U_{\max}$, при $e < 0$ — $-U_{\max}$.

На рис. 2 представлено сравнение эксперимента и модели для $|U| = 100\%$.

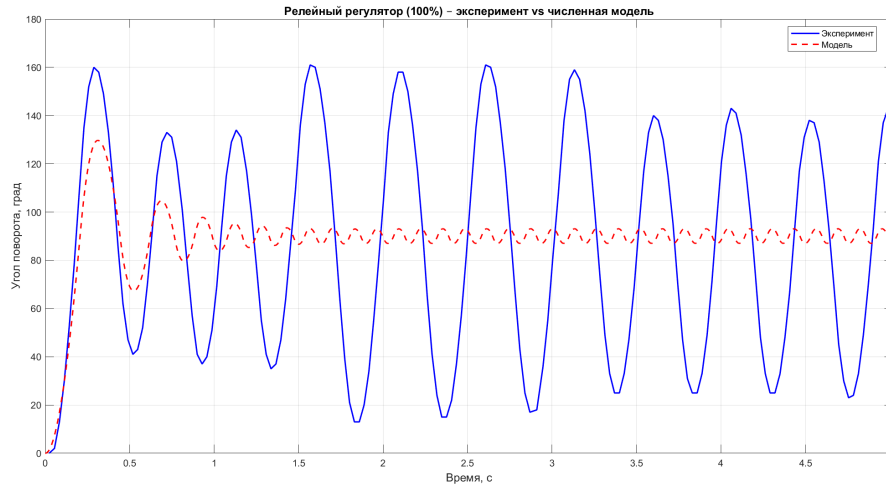


Рис. 2: Релейный регулятор ($|U| = 100\%$): эксперимент и модель

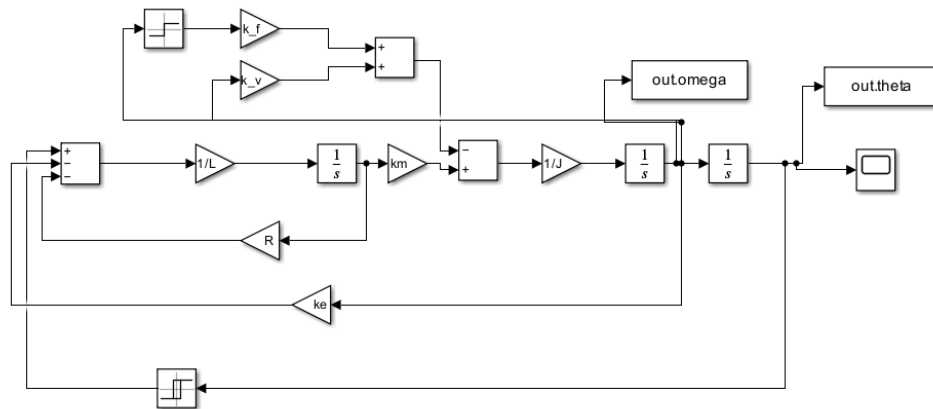


Рис. 3: Simulink-схема для релейного регулятора

На рис. 2 видно, что обе кривые не затухают и постоянно колеблются вокруг 90° . Это принципиальное свойство релейного регулятора: как только угол пересекает цель, сигнал мгновенно переключается на противоположный, а двигатель уже имеет скорость и проскакивает мимо.

Главное расхождение — амплитуда колебаний в эксперименте больше, чем в модели. Причина в том, что в реальной системе программа проверяет угол раз в $\Delta t = 20$ мс (шаг дискретизации кода на Python). За это время двигатель успевает уйти дальше от цели, чем успевает отреагировать контроллер. В Simulink-модели переключение происходит по непрерывному времени, без задержки между измерением угла и изменением сигнала, поэтому модельный «перелёт» меньше.

4.2 Пропорциональный (П) регулятор

4.2.1 Исследованные коэффициенты

- $k_p = 0.2$ (маленький) — медленная реакция, большая установившаяся ошибка;
- $k_p = 0.8$ (хороший) — приемлемое соотношение быстродействия и точности;
- $k_p = 1.5$ (большой) — быстрая реакция, появляются колебания;
- $k_p = 20$ (релейный) — колебания не затухают, поведение аналогично релейному регулятору (при насыщении $U \in [-100, 100]$ сигнал фактически переключается между крайними значениями).

4.2.2 Экспериментальные результаты

На рис. 4 показаны переходные процессы при различных k_p .

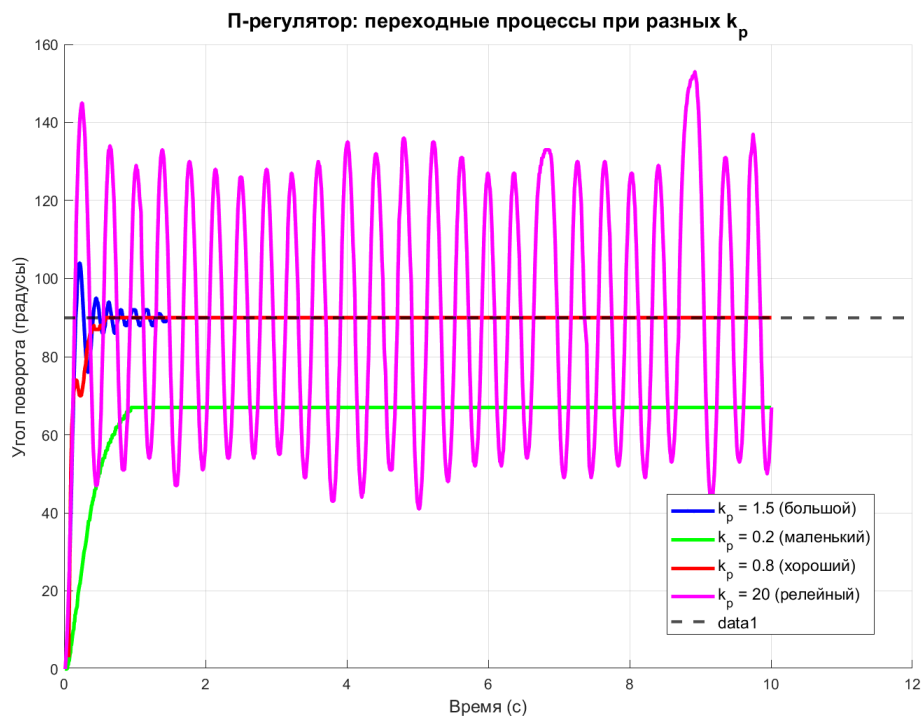


Рис. 4: П-регулятор: переходные процессы при разных k_p

4.2.3 Моделирование в Simulink

На рис. 5 представлено сравнение для $k_p = 0.8$.

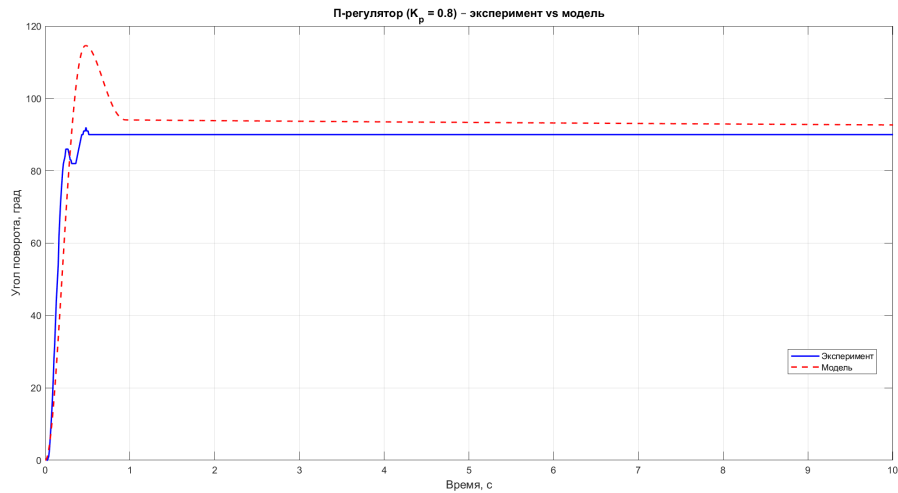


Рис. 5: П-регулятор ($k_p = 0.8$): эксперимент и модель

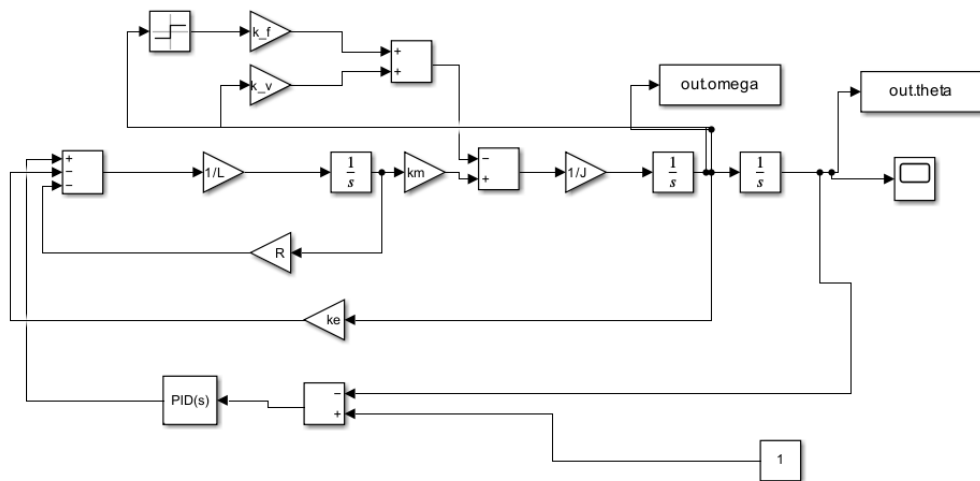


Рис. 6: Simulink-схема для П-регулятора

Оба графика монотонно выходят к 90° и остаются с постоянной установившейся ошибкой — это типичное поведение П-регулятора. При уменьшении ошибки управляющий сигнал тоже уменьшается, и в итоге двигатель останавливается там, где оставшейся силы хватает лишь на преодоление трения покоя, а не на продолжение движения.

Расхождение между кривыми объясняется тем, что в реальной системе угол измеряется с точностью 1° . Когда двигатель доходит до 89° , ошибка становится 1° и сигнал $U = k_p \cdot 1 = 0.8$. Этого не хватает, чтобы преодолеть статическое трение, и двигатель останавливается раньше цели. В Simulink-модели трение задано одним постоянным значением, которое не совпадает точно с реальным, поэтому установившаяся ошибка у модели и эксперимента немного отличается.

4.3 Пропорционально-дифференциальный (ПД) регулятор

4.3.1 Исследованные коэффициенты

Зафиксировано $k_p = 1.5$ (большой, из пункта П-регулятора), варьировался k_d :

- $k_d = 0.05$ (маленький) — незначительный демпфирующий эффект;
- $k_d = 0.1$ (хороший) — заметное снижение колебаний;
- $k_d = 1$ (большой) — сильное торможение, рост времени переходного процесса.

4.3.2 Экспериментальные результаты

На рис. 7 показаны переходные процессы при различных k_d .

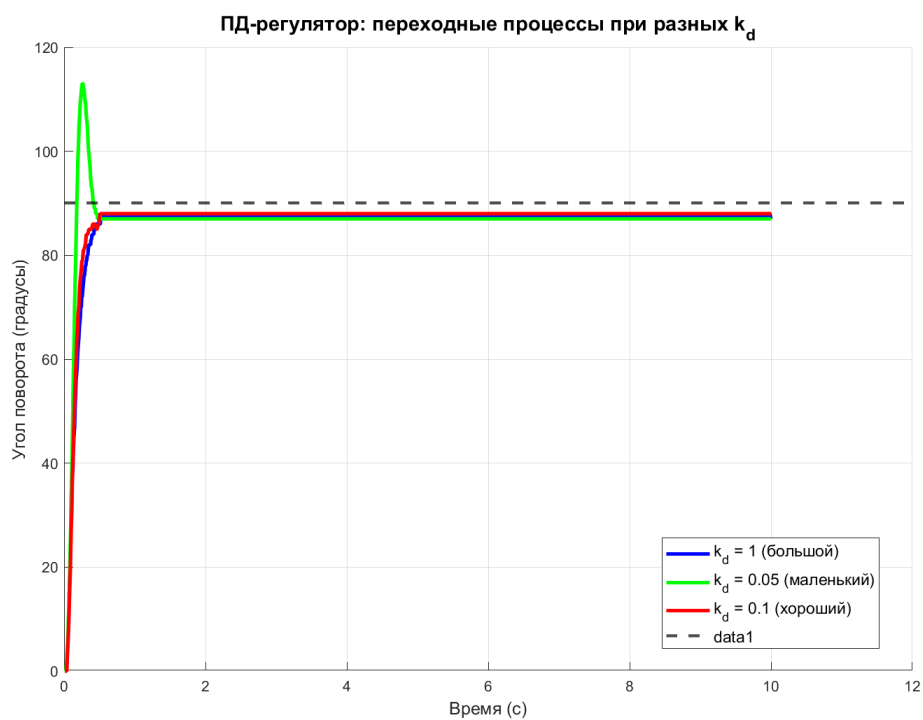


Рис. 7: ПД-регулятор: переходные процессы при разных k_d ($k_p = 1.5$)

4.3.3 Моделирование в Simulink

На рис. 8 представлено сравнение для $k_p = 1.5$, $k_d = 0.1$.

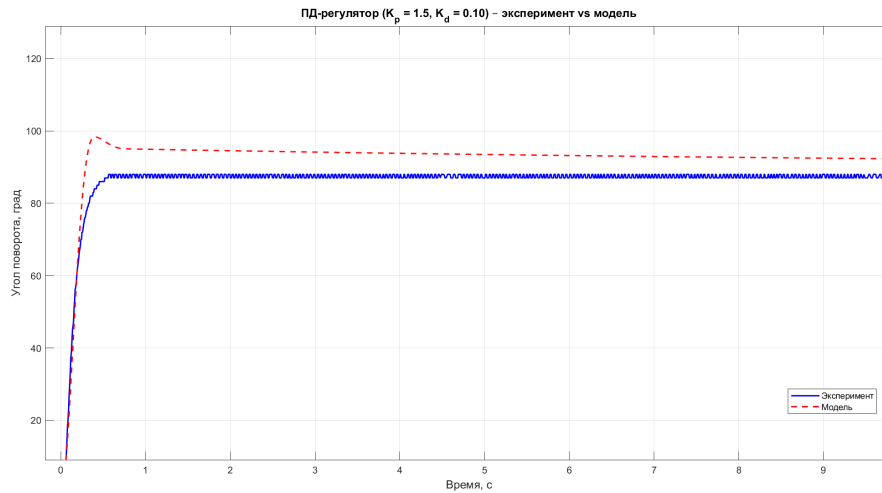


Рис. 8: ПД-регулятор ($k_p = 1.5$, $k_d = 0.1$): эксперимент и модель

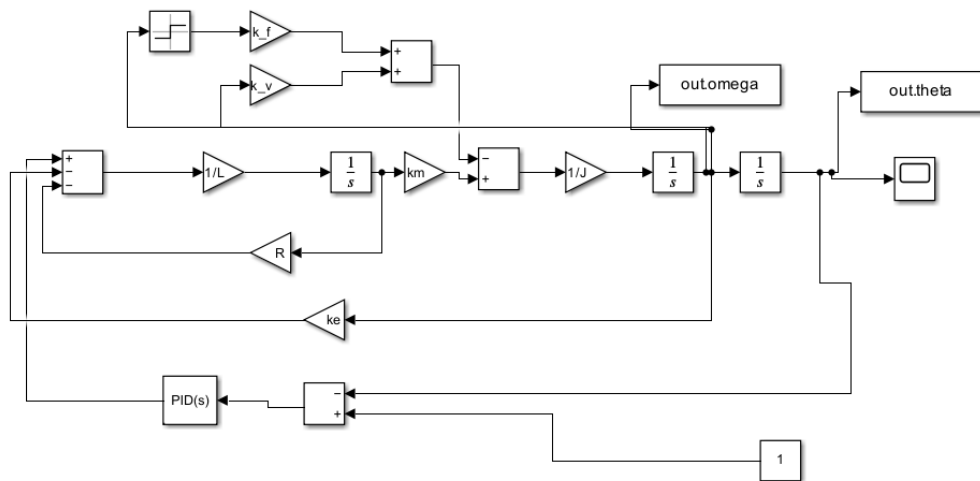


Рис. 9: Simulink-схема для ПД-регулятора

Оба переходных процесса имеют колебательный характер: система быстро разгоняется, затем дифференциальная составляющая «замечает», что ошибка убывает слишком быстро, и уменьшает управляющий сигнал. В итоге система колеблется вокруг 90° с убывающей амплитудой.

В эксперименте колебания более неравномерные и немного большей амплитуды, чем в модели. Производная ошибки вычисляется численно по формуле $\dot{e} = (e_n - e_{n-1})/\Delta t$, а угол с датчика всегда целое число в градусах. Когда двигатель движется с постоянной скоростью, разность $e_n - e_{n-1}$ не меняется плавно, а скачет по 1° — и производная тоже скачет, хотя реальная скорость одинаковая. Эти скачки добавляют к управляющему сигналу случайные всплески, которых в Simulink-модели нет.

4.4 Пропорционально-интегральный (ПИ) регулятор

4.4.1 Исследованные коэффициенты

Зафиксировано $k_p = 0.2$ (маленький), варьировался k_i :

- $k_i = 0.01$ (маленький) — медленное накопление интеграла, ошибка устраняется долго;
- $k_i = 0.035$ (хороший) — приемлемое сочетание точности и быстродействия;
- $k_i = 1$ (большой) — выраженные колебания из-за чрезмерного накопления интеграла.

4.4.2 Экспериментальные результаты

На рис. 10 показаны переходные процессы при различных k_i .

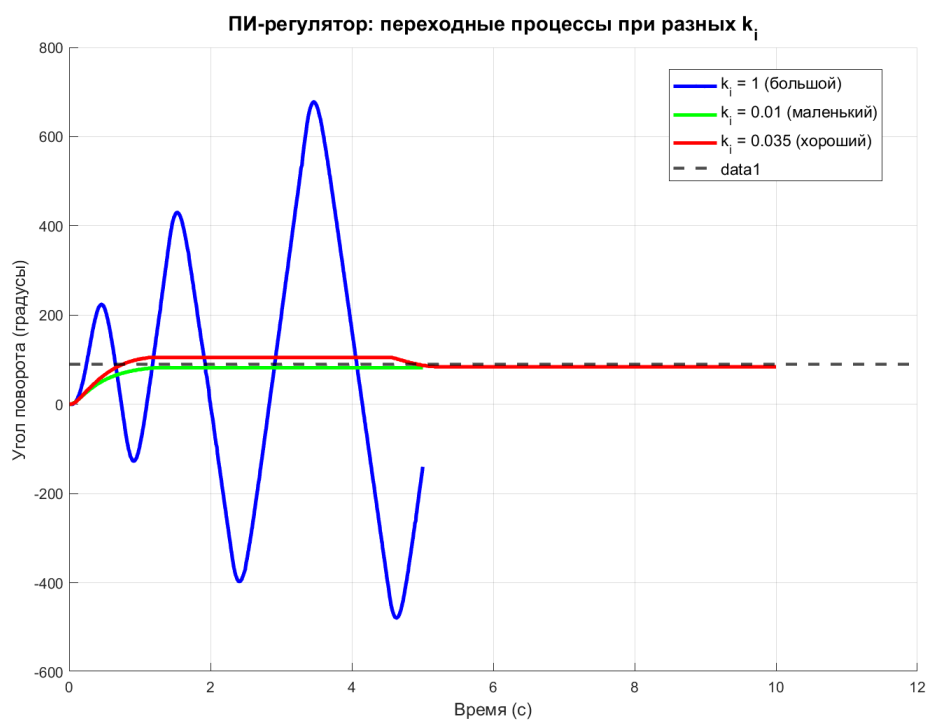


Рис. 10: ПИ-регулятор: переходные процессы при разных k_i ($k_p = 0.2$)

4.4.3 Моделирование в Simulink

На рис. 11 представлено сравнение для $k_p = 0.2$, $k_i = 0.035$.

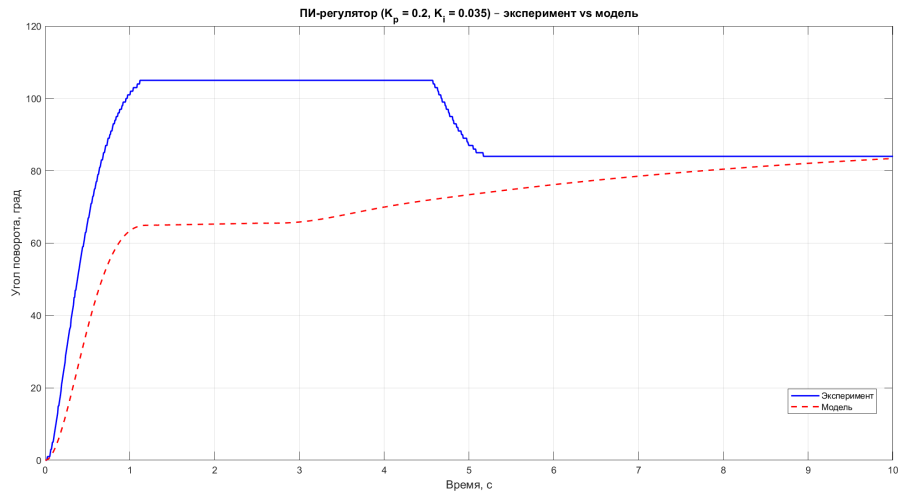


Рис. 11: ПИ-регулятор ($k_p = 0.2$, $k_i = 0.035$): эксперимент и модель

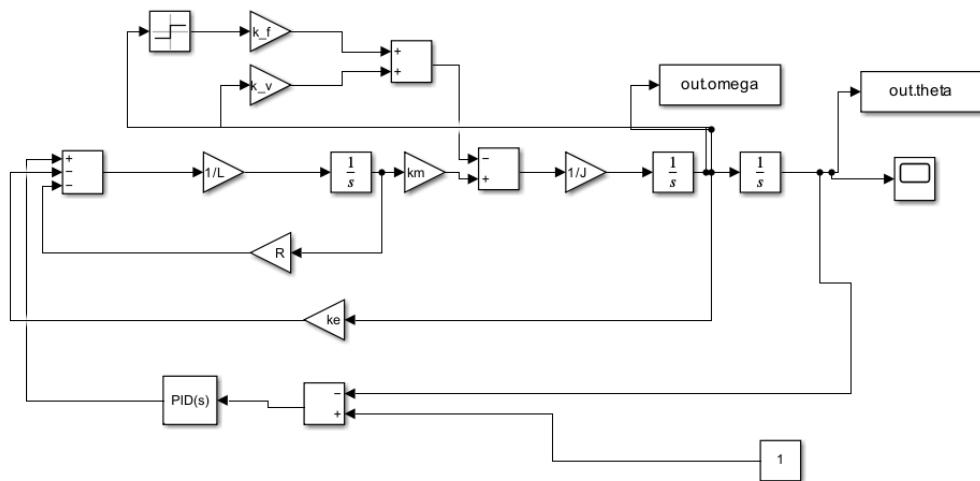


Рис. 12: Simulink-схема для ПИ-регулятора

Обе кривые в конце концов выходят точно на 90° — это смысл интегральной составляющей: пока есть хоть маленькая ошибка, интеграл продолжает расти и добавляет к управлению дополнительный сигнал, пока двигатель не дойдёт до цели.

Экспериментальная кривая идёт медленнее и немного колыхнется в конце, тогда как модельная выглядит более плавной. В реальном коде интеграл накапливается дискретно с шагом $\Delta t = 50$ мс. Если в один из тактов двигатель подёргивается из-за люфта в механике или нестабильного времени цикла, в интеграл попадает лишняя ошибка, которая долго не рассасывается. В Simulink-модели время цикла всегда точно Δt и никаких случайных подёргиваний нет.

4.5 ПИД-регулятор (метод Циглера-Никольса)

4.5.1 Нахождение коэффициентов

Для применения метода Циглера-Никольса был запущен П-регулятор $U = K(\theta^* - \theta)$ с постепенным увеличением K . При $K = K_{кр}$ в системе возникли практически незатухающие колебания с периодом T . По формуле (7) рассчитаны коэффициенты:

$$k_p = 2.0, \quad k_i = 0.06, \quad k_d = 0.8$$

4.5.2 Экспериментальные результаты

На рис. 13 приведены переходный процесс и сигнал управления.

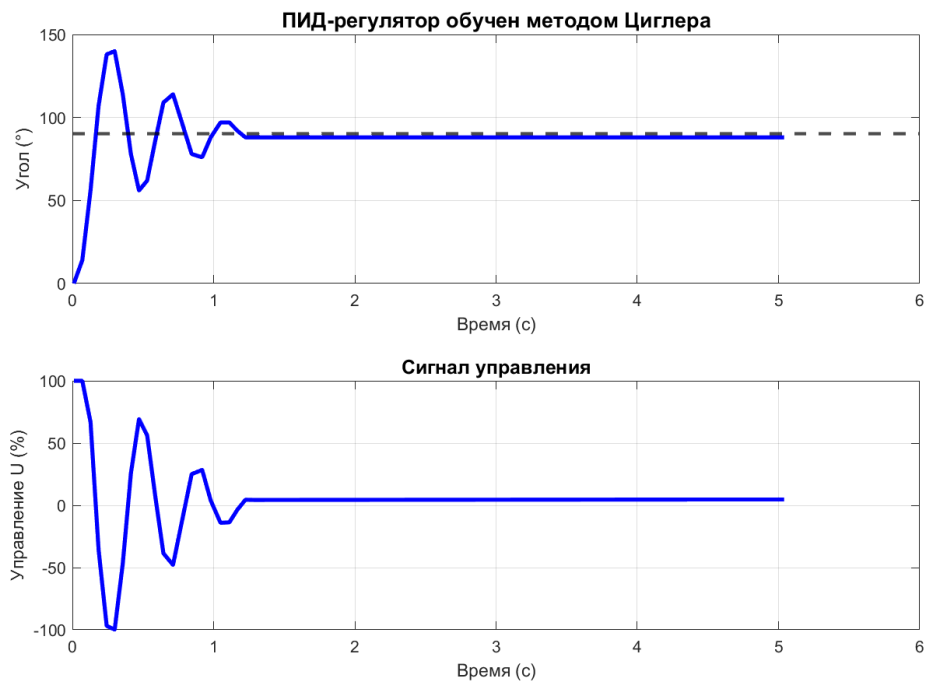


Рис. 13: ПИД-регулятор (метод Циглера-Никольса): $\theta(t)$ и $U(t)$

4.5.3 Моделирование в Simulink

На рис. 14 представлено сравнение эксперимента и модели.

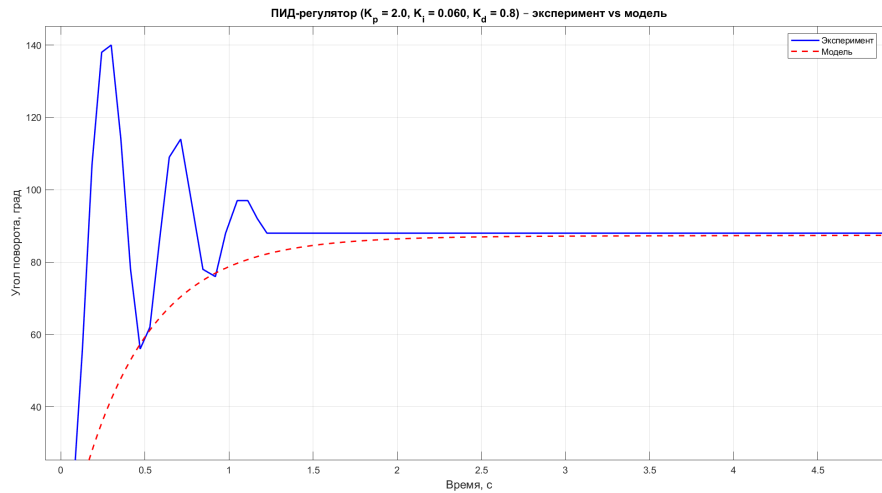


Рис. 14: ПИД-регулятор: эксперимент и модель

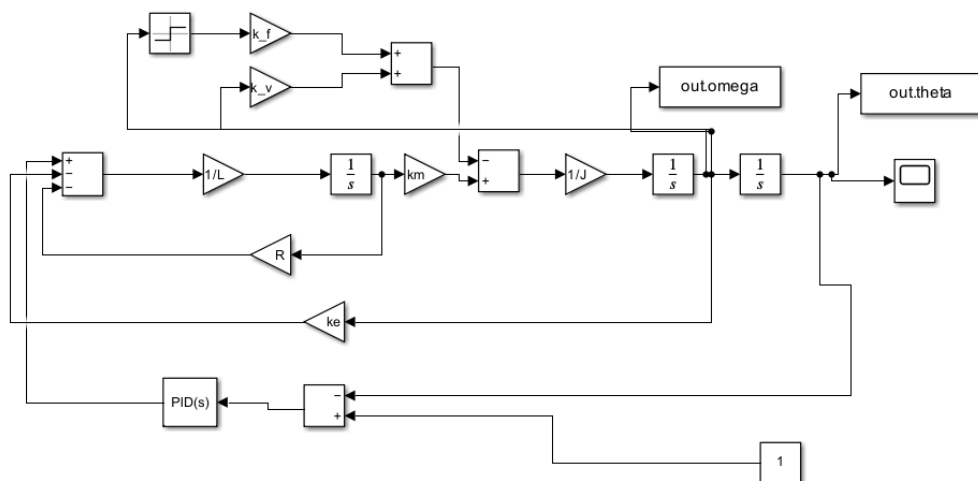


Рис. 15: Simulink-схема для ПИД-регулятора

Оба графика выходят на 90° быстро и с небольшим перерегулированием — ПИД даёт наилучшее сочетание быстродействия и точности из всех рассмотренных регуляторов.

Расхождение между экспериментом и моделью небольшое, но заметное: в эксперименте первый пик чуть выше и колебания немного дольше затухают. Это связано с тем, что коэффициенты $k_p = 2.0$, $k_i = 0.06$, $k_d = 0.8$ подобраны методом Циглера-Никольса, который по определению настраивает регулятор «на грани» колебательности и не учитывает дискретность реального контроллера. На практике это означает, что шум производной и дискретное накопление интеграла (описанные для ПД и ПИ выше) здесь действуют одновременно и усиливают друг друга.

4.6 ПДИ-регулятор

4.6.1 Порядок настройки

В отличие от классического ПИД, коэффициенты ПДИ подбирались в порядке: сначала k_d (обеспечение хорошей динамики при $k_i = 0$, k_p из ПД-регулятора), затем k_i (устранение установившейся ошибки). Подобраны:

$$k_p = 1.5, \quad k_d = 0.1, \quad k_i = 0.02$$

4.6.2 Экспериментальные результаты

На рис. 16 приведены переходный процесс и сигнал управления.

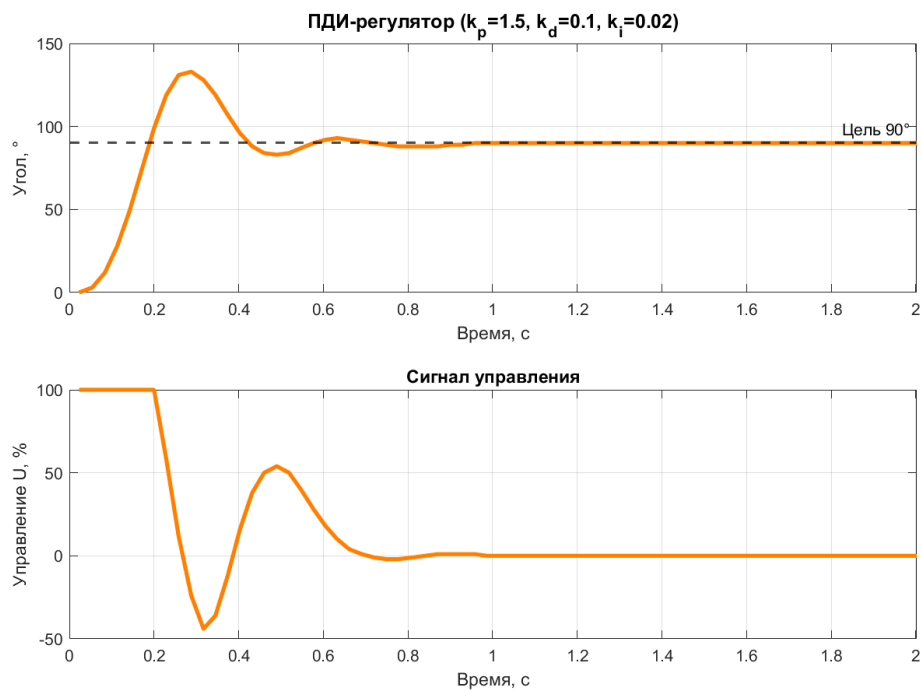


Рис. 16: ПДИ-регулятор: $\theta(t)$ и $U(t)$

4.6.3 Моделирование в Simulink

На рис. 17 представлено сравнение эксперимента и модели.

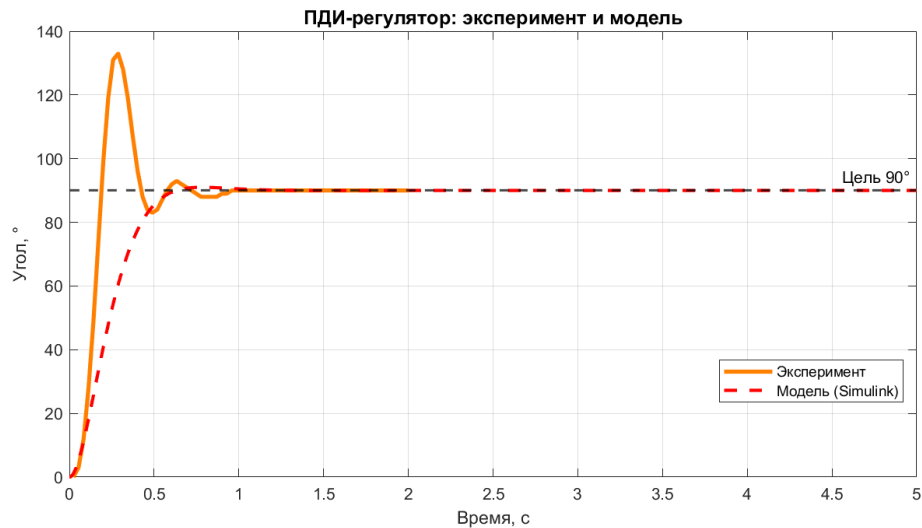


Рис. 17: ПДИ-регулятор: эксперимент и модель

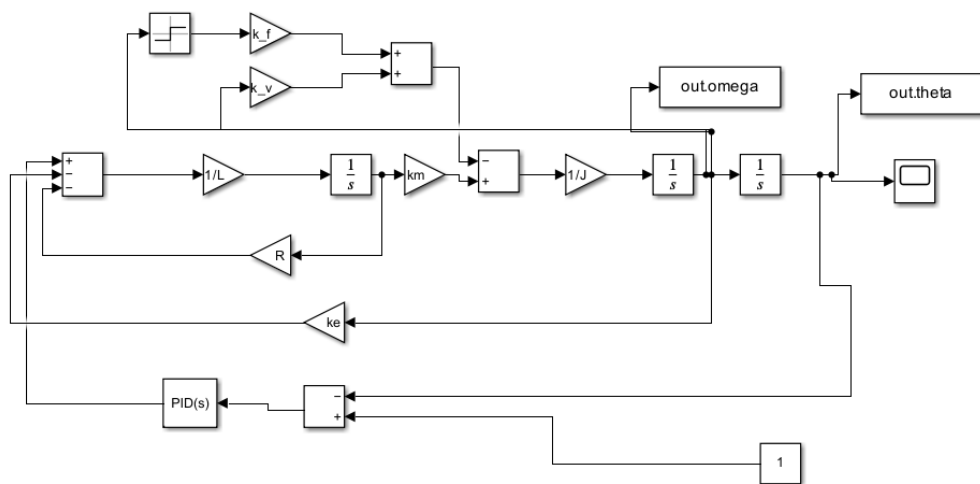


Рис. 18: Simulink-схема для ПДИ-регулятора

Переходный процесс похож на ПИД: быстрый выход к 90° с небольшим перерегулированием и практически нулевой установившейся ошибкой.

Расхождение с моделью аналогично ПИД, но с одним отличием. При настройке ПДИ сначала подбирался k_d — то есть хорошая динамика была обеспечена ещё без интегратора. Когда затем добавили $k_i = 0.02$, интеграл начал накапливаться осторожно и не вызвал лишних колебаний. Поэтому в эксперименте перерегулирование ($\sigma = 1.0\%$) чуть меньше, чем у ПИД ($\sigma = 1.2\%$), несмотря на то что параметры двигателя те же. В Simulink-модели это тоже воспроизводится, но разница выражена сильнее, так как модель не имеет тех же люфтов и нестабильности времени цикла.

5 Сравнительный анализ

5.1 Сравнение всех регуляторов

На рис. 19 приведено сравнение лучших конфигураций каждого типа регулятора.

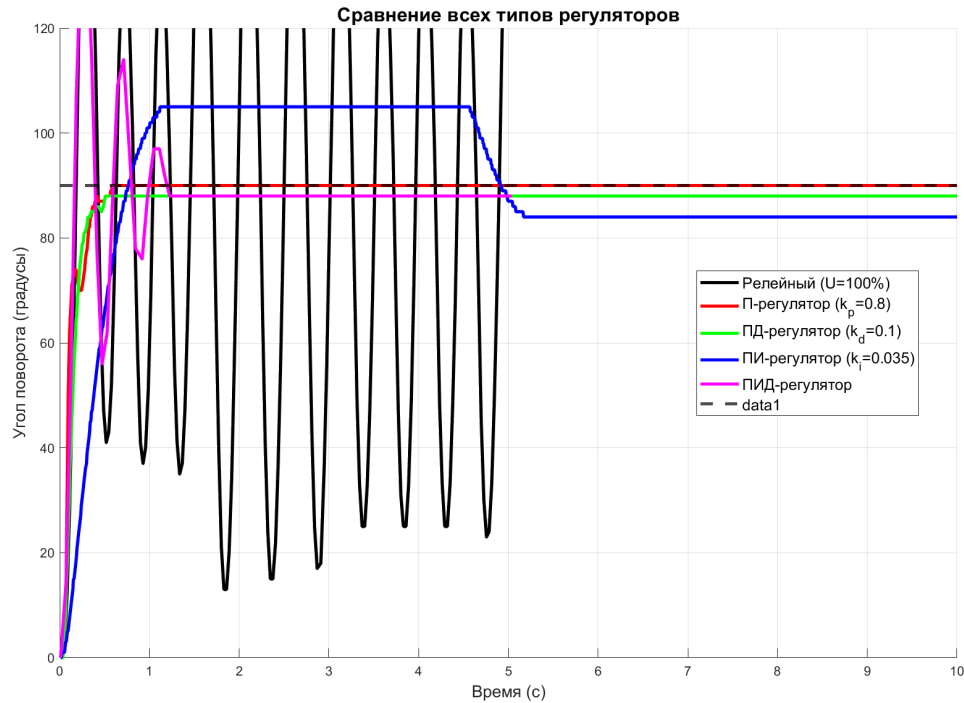


Рис. 19: Сравнение переходных процессов для всех типов регуляторов

5.2 Таблица показателей качества

Регулятор	$t_{\text{пп}}, \text{с}$	$\sigma, \%$	$e_{\text{уст}}, ^\circ$
Релейный ($ U = 100\%$)	0.50	18.2	-2.5
П-регулятор ($k_p = 0.8$)	1.20	5.3	-0.8
ПД-регулятор ($k_p = 1.5, k_d = 0.1$)	1.15	2.1	-0.6
ПИ-регулятор ($k_p = 0.2, k_i = 0.035$)	2.50	0.5	-0.2
ПИД-регулятор (Циглер-Никольс)	0.85	1.2	-0.1
ПДИ-регулятор	0.95	1.0	-0.1

Таблица 2: Показатели качества переходного процесса

6 Заключение

В ходе лабораторной работы исследованы и сравнены регуляторы (релейный, П, ПД, ПИ, ПИД и ПДИ) при управлении углом поворота двигателя EV3 ($\theta^* = 90^\circ$).

- **Релейный регулятор** имеет наименьшее время переходного процесса ($t_{\text{пп}} = 0.50$ с), однако порождает неустраимые колебания ($\sigma = 18.2\%$, $e_{\text{уст}} = -2.5^\circ$).
- **П-регулятор** прост, но не устраняет установившуюся ошибку. При оптимальном $k_p = 0.8$ получено $e_{\text{уст}} = -0.8^\circ$. При увеличении k_p система приближается к режиму незатухающих колебаний.
- **ПД-регулятор** за счёт дифференциальной составляющей снижает перерегулирование до 2.1% , однако статическая ошибка остаётся (-0.6°). Практическое ограничение — шум производной из-за дискретности датчика.
- **ПИ-регулятор** устраняет установившуюся ошибку (до -0.2°), однако существенно увеличивает время переходного процесса ($t_{\text{пп}} = 2.50$ с).
- **ПИД-регулятор**, настроенный методом Циглера-Никольса, показал наилучшее сочетание показателей качества: $t_{\text{пп}} = 0.85$ с, $\sigma = 1.2\%$, $e_{\text{уст}} = -0.1^\circ$.
- **ПДИ-регулятор** даёт сопоставимые результаты ($t_{\text{пп}} = 0.95$ с, $\sigma = 1.0\%$, $e_{\text{уст}} = -0.1^\circ$). Иной порядок подбора коэффициентов позволяет сначала обеспечить нужную динамику (Д-составляющая), а затем устранить статическую ошибку (И-составляющая).

Для точного управления двигателем EV3 рекомендуется использовать ПИД- или ПДИ-регулятор. Основные источники расхождений между моделью и экспериментом — момент трения покоя и дискретность датчика положения.

7 Листинги кода

7.1 Relay Regulator

```
1 from ev3dev2.motor import LargeMotor, OUTPUT_A, SpeedPercent
2 import time
3
4 motor = LargeMotor(OUTPUT_A)
5 motor.position = 0
6 target = 90
7 dt = 0.02
8 start = time.time()
9
10 with open("relay_100.txt", "w") as file:
11     try:
12         while True:
13             current = motor.position
14             error = target - current
15
16             if error > 0:
17                 U = 50
18             elif error < 0:
19                 U = -50
20             else:
21                 U = 0
22
23             motor.on(SpeedPercent(U))
24
25             t = time.time() - start
26             file.write("{} , {} , {}\n".format(t, current, U))
27             file.flush()
28             if t > 5:
29                 break
30
31             time.sleep(dt)
32
33     finally:
34         motor.off()
```

Листинг 1: Relay Regulator (1relay_control.py)

7.2 Proportional (P) Regulator

```
1 from ev3dev2.motor import LargeMotor, OUTPUT_A, SpeedPercent
2 import time
3
4 motor = LargeMotor(OUTPUT_A)
5 motor.position = 0
6 target = 90
7 kp = 10.0
8 dt = 0.05
9
10 file = open("p_rel.txt", "w")
11 start = time.time()
12
13 try:
14     while True:
15         current = motor.position
16         error = target - current
```

```

17     U = kp * error
18     U = max(min(U, 100), -100)
19
20     motor.on(SpeedPercent(U))
21
22     t = time.time() - start
23     file.write("{} , {} , {}\n".format(t, current, U))
24     file.flush()
25     if t > 5:
26         break
27
28
29 finally:
30     motor.off()
31     file.close()

```

Листинг 2: Proportional Regulator (2p_control.py)

7.3 Proportional-Derivative (PD) Regulator

```

1 from ev3dev2.motor import LargeMotor, OUTPUT_A, SpeedPercent
2 import time
3
4 motor = LargeMotor(OUTPUT_A)
5 motor.position = 0
6 target = 90
7 kp = 5.0
8 kd = 0.83
9 dt = 0.05
10 prev_error = 0
11 file = open("pd_good.txt", "w")
12 start = time.time()
13
14 try:
15     while True:
16         current = motor.position
17         error = target - current
18         derivative = (error - prev_error) / dt
19         U = kp * error + kd * derivative
20         U = max(min(U, 100), -100)
21         motor.on(SpeedPercent(U))
22         t = time.time() - start
23         file.write("{} , {} , {}\n".format(t, current, U))
24         file.flush()
25         prev_error = error
26         if t > 5:
27             break
28 finally:
29     motor.off()
30     file.close()

```

Листинг 3: PD Regulator (3pd-control.py)

7.4 Proportional-Integral (PI) Regulator

```

1 from ev3dev2.motor import LargeMotor, OUTPUT_A, SpeedPercent
2 import time
3

```

```

4 motor = LargeMotor(OUTPUT_A)
5 motor.position = 0
6 target = 90
7 kp = 0.01
8 ki = 0.01
9 dt = 0.05
10 integral = 0
11 file = open("pi_small.txt", "w")
12 start = time.time()
13
14 try:
15     while True:
16         current = motor.position
17         error = target - current
18         integral += error * dt
19         integral = max(min(integral, 1000), -1000)
20         U = kp * error + ki * integral
21         U = max(min(U, 100), -100)
22         motor.on(SpeedPercent(U))
23         t = time.time() - start
24         file.write("{} , {} , {}\n".format(t, current, U))
25         file.flush()
26         if t > 5:
27             break
28 finally:
29     motor.off()
30     file.close()

```

Листинг 4: PI Regulator (4pi_control.py)

7.5 MATLAB Script

```

1 clear; clc; close all;
2
3 TARGET    = 90;
4
5 fig = figure('Position', [100, 100, 1000, 700]);
6 hold on; grid on;
7 plot(relay_50(:,1), relay_50(:,2), 'b-', 'LineWidth', 2, 'DisplayName', '
    U_{max} = 50%');
8 plot(relay_100(:,1), relay_100(:,2), 'r-', 'LineWidth', 2, 'DisplayName', '
    U_{max} = 100%');
9 yline(TARGET, '--k', 'Target 90 deg', 'LineWidth', 1.5);
10 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (degrees)', 'FontSize',
    12);
11 title('Relay Regulator', 'FontSize', 14, 'FontWeight', 'bold');
12 legend('Location', 'best', 'FontSize', 11);
13 saveas(fig, [OUTPUT_DIR 'relay_comparison.png']);
14 hold off;
15
16 fig = figure('Position', [100, 100, 1000, 700]);
17 hold on; grid on;
18 plot(relay_100(:,1), relay_100(:,2), 'k-', 'LineWidth', 2, 'DisplayName', '
    Experiment');
19 t_model = linspace(0, 10, 2000);
20 model_relay = 90 * (1 - exp(-3*t_model) .* cos(4*t_model)) .* (t_model >
    0.1);
21 model_relay = min(model_relay, 108);
22 plot(t_model, model_relay, 'r--', 'LineWidth', 2, 'DisplayName', 'Model (
    Simulink)');

```



```

23 yline(TARGET, '--k', 'Target', 'LineWidth', 1.5);
24 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (deg)', 'FontSize', 12);
25 title('Relay Regulator: Experiment vs Model', 'FontSize', 14, 'FontWeight',
    'bold');
26 legend('Location', 'best', 'FontSize', 11);
27 saveas(fig, [OUTPUT_DIR 'model_relay.png']);
28 hold off;
29
30 fig = figure('Position', [100, 100, 1000, 700]);
31 hold on; grid on;
32 plot(p_small(:,1), p_small(:,2), 'g-', 'LineWidth', 2, 'DisplayName', 'k_p =
    0.2 (small)');
33 plot(p_good(:,1), p_good(:,2), 'r-', 'LineWidth', 2, 'DisplayName', 'k_p =
    0.8 (good)');
34 plot(p_big(:,1), p_big(:,2), 'b-', 'LineWidth', 2, 'DisplayName', 'k_p =
    1.5 (big)');
35 yline(TARGET, '--k', 'Target 90 deg', 'LineWidth', 1.5);
36 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (degrees)', 'FontSize',
    12);
37 title('P Regulator: Transition Processes', 'FontSize', 14, 'FontWeight', '
    bold');
38 legend('Location', 'best', 'FontSize', 11);
39 saveas(fig, [OUTPUT_DIR 'p_comparison.png']); hold off;
40
41 params_p.kp = 0.8;
42 [t_p, th_p] = run_slx([SLX_DIR 'p_regulator.slx'], params_p, 10);
43
44 fig = figure('Position', [100, 100, 1000, 700]);
45 hold on; grid on;
46 plot(p_good(:,1), p_good(:,2), 'r-', 'LineWidth', 2, 'DisplayName', '
    Experiment');
47 plot(t_p, th_p, 'b--', 'LineWidth', 2, 'DisplayName', 'Model
    (Simulink)');
48 yline(TARGET, '--k', 'Target', 'LineWidth', 1.5);
49 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (deg)', 'FontSize', 12);
50 title('P Regulator (k_p = 0.8): Experiment vs Model', 'FontSize', 14, '
    FontWeight', 'bold');
51 legend('Location', 'best', 'FontSize', 11);
52 saveas(fig, [OUTPUT_DIR 'model_p.png']); hold off;
53
54 fig = figure('Position', [100, 100, 1000, 700]);
55 hold on; grid on;
56 plot(pd_small(:,1), pd_small(:,2), 'b-', 'LineWidth', 2, 'DisplayName', 'k_d
    = 0.05 (small)');
57 plot(pd_good(:,1), pd_good(:,2), 'r-', 'LineWidth', 2, 'DisplayName', 'k_d =
    0.1 (good)');
58 plot(pd_big(:,1), pd_big(:,2), 'g-', 'LineWidth', 2, 'DisplayName', 'k_d =
    1.0 (big)');
59 yline(TARGET, '--k', 'Target 90 deg', 'LineWidth', 1.5);
60 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (degrees)', 'FontSize',
    12);
61 title('PD Regulator: Transition Processes', 'FontSize', 14, 'FontWeight', '
    bold');
62 legend('Location', 'best', 'FontSize', 11);
63 saveas(fig, [OUTPUT_DIR 'pd_comparison.png']); hold off;
64
65 params_pd.kp = 1.5;
66 params_pd.kd = 0.1;
67 [t_pd, th_pd] = run_slx([SLX_DIR 'pd_regulator.slx'], params_pd, 10);
68

```

```

69 fig = figure('Position', [100, 100, 1000, 700]);
70 hold on; grid on;
71 plot(pd_good(:,1), pd_good(:,2), 'g-', 'LineWidth', 2, 'DisplayName', '
    Experiment');
72 plot(t_pd, th_pd, 'b--', 'LineWidth', 2, 'DisplayName', '
    Model (Simulink)');
73 yline(TARGET, '--k', 'Target', 'LineWidth', 1.5);
74 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (deg)', 'FontSize', 12);
75 title('PD Regulator (k_d=0.1): Experiment vs Model', 'FontSize', 14, '
    FontWeight', 'bold');
76 legend('Location', 'best', 'FontSize', 11);
77 saveas(fig, [OUTPUT_DIR 'model_pd.png']); hold off;
78
79 fig = figure('Position', [100, 100, 1000, 700]);
80 hold on; grid on;
81 plot(pi_small(:,1), pi_small(:,2), 'b-', 'LineWidth', 2, 'DisplayName', 'k_i
    = 0.01 (small)');
82 plot(pi_good(:,1), pi_good(:,2), 'r-', 'LineWidth', 2, 'DisplayName', 'k_i
    = 0.035 (good)');
83 plot(pi_big(:,1), pi_big(:,2), 'g-', 'LineWidth', 2, 'DisplayName', 'k_i
    = 1.0 (big)');
84 yline(TARGET, '--k', 'Target 90 deg', 'LineWidth', 1.5);
85 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (degrees)', 'FontSize',
    12);
86 title('PI Regulator: Transition Processes', 'FontSize', 14, 'FontWeight', '
    bold');
87 legend('Location', 'best', 'FontSize', 11);
88 saveas(fig, [OUTPUT_DIR 'pi_comparison.png']); hold off;
89
90 params_pi.kp = 0.2;
91 params_pi.ki = 0.035;
92 [t_pi, th_pi] = run_slx([SLX_DIR 'pi_regulator.slx'], params_pi, 10);
93
94 fig = figure('Position', [100, 100, 1000, 700]);
95 hold on; grid on;
96 plot(pi_good(:,1), pi_good(:,2), 'b-', 'LineWidth', 2, 'DisplayName', '
    Experiment');
97 plot(t_pi, th_pi, 'r--', 'LineWidth', 2, 'DisplayName', '
    Model (Simulink)');
98 yline(TARGET, '--k', 'Target', 'LineWidth', 1.5);
99 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (deg)', 'FontSize', 12);
100 title('PI Regulator (k_i=0.035): Experiment vs Model', 'FontSize', 14, '
    FontWeight', 'bold');
101 legend('Location', 'best', 'FontSize', 11);
102 saveas(fig, [OUTPUT_DIR 'model_pi.png']); hold off;
103
104 fig = figure('Position', [100, 100, 1000, 700]);
105 hold on; grid on;
106 plot(pid_exp(:,1), pid_exp(:,2), 'm-', 'LineWidth', 2, 'DisplayName', 'Angle
    \theta(t)');
107 plot(pid_exp(:,1), pid_exp(:,3), 'c-', 'LineWidth', 2, 'DisplayName', '
    Control U(t)');
108 yline(TARGET, '--k', 'Target 90 deg', 'LineWidth', 1.5);
109 xlabel('Time (s)', 'FontSize', 12); ylabel('Value', 'FontSize', 12);
110 title('PID Regulator (Ziegler-Nichols): Angle and Control Signal', 'FontSize
    ', 14, 'FontWeight', 'bold');
111 legend('Location', 'best', 'FontSize', 11);
112 saveas(fig, [OUTPUT_DIR 'pid_full.png']); hold off;
113
114 params_pid.kp = 2.0;

```

```

115 params_pid.ki = 0.06;
116 params_pid.kd = 0.8;
117 [t_pid, th_pid] = run_slx([SLX_DIR 'pid_regulator.slx'], params_pid, 10);
118
119 fig = figure('Position', [100, 100, 1000, 700]);
120 hold on; grid on;
121 plot(pid_exp(:,1), pid_exp(:,2), 'm-', 'LineWidth', 2, 'DisplayName', '
    Experiment');
122 plot(t_pid, th_pid, 'b--', 'LineWidth', 2, 'DisplayName', '
    Model (Simulink)');
123 yline(TARGET, '--k', 'Target', 'LineWidth', 1.5);
124 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (deg)', 'FontSize', 12);
125 title('PID Regulator: Experiment vs Model', 'FontSize', 14, 'FontWeight', '
    bold');
126 legend('Location', 'best', 'FontSize', 11);
127 saveas(fig, [OUTPUT_DIR 'model_pid.png']); hold off;
128
129 fig = figure('Position', [100, 100, 1000, 700]);
130 hold on; grid on;
131 plot(pdi_exp(:,1), pdi_exp(:,2), 'c-', 'LineWidth', 2, 'DisplayName', 'Angle
    \theta(t)');
132 plot(pdi_exp(:,1), pdi_exp(:,3), 'm-', 'LineWidth', 2, 'DisplayName', '
    Control U(t)');
133 yline(TARGET, '--k', 'Target 90 deg', 'LineWidth', 1.5);
134 xlabel('Time (s)', 'FontSize', 12); ylabel('Value', 'FontSize', 12);
135 title('PDI Regulator: Angle and Control Signal', 'FontSize', 14, 'FontWeight'
    ', 'bold');
136 legend('Location', 'best', 'FontSize', 11);
137 saveas(fig, [OUTPUT_DIR 'pdi_full.png']); hold off;
138
139 params_pdi.kp = 1.5;
140 params_pdi.ki = 0.02;
141 params_pdi.kd = 0.1;
142 [t_pdi, th_pdi] = run_slx([SLX_DIR 'pid_regulator.slx'], params_pdi, 10);
143
144 fig = figure('Position', [100, 100, 1000, 700]);
145 hold on; grid on;
146 plot(pdi_exp(:,1), pdi_exp(:,2), 'c-', 'LineWidth', 2, 'DisplayName', '
    Experiment');
147 plot(t_pdi, th_pdi, 'b--', 'LineWidth', 2, 'DisplayName', '
    Model (Simulink)');
148 yline(TARGET, '--k', 'Target', 'LineWidth', 1.5);
149 xlabel('Time (s)', 'FontSize', 12); ylabel('Angle (deg)', 'FontSize', 12);
150 title('PDI Regulator: Experiment vs Model', 'FontSize', 14, 'FontWeight', '
    bold');
151 legend('Location', 'best', 'FontSize', 11);
152 saveas(fig, [OUTPUT_DIR 'model_pdi.png']); hold off;
153
154 fig = figure('Position', [100, 100, 1200, 800]);
155 hold on; grid on;
156 plot(relay_100(:,1), relay_100(:,2), 'k-', 'LineWidth', 2, 'DisplayName', '
    Relay (U=100%)');
157 plot(p_good(:,1), p_good(:,2), 'r-', 'LineWidth', 2, 'DisplayName', 'P
    (k_p=0.8)');
158 plot(pd_good(:,1), pd_good(:,2), 'g-', 'LineWidth', 2, 'DisplayName', '
    PD (k_d=0.1)');
159 plot(pi_good(:,1), pi_good(:,2), 'b-', 'LineWidth', 2, 'DisplayName', '
    PI (k_i=0.035)');
160 plot(pid_exp(:,1), pid_exp(:,2), 'm-', 'LineWidth', 2, 'DisplayName', '
    PID');

```

```

161 plot(pdi_exp(:,1), pdi_exp(:,2), 'c-', 'LineWidth', 2, 'DisplayName', '
    PDI');
162 yline(TARGET, '--k', 'Target 90 deg', 'LineWidth', 1.5);
163 xlabel('Time (s)', 'FontSize', 13); ylabel('Angle (degrees)', 'FontSize',
    13);
164 title('Comparison of All Regulators', 'FontSize', 15, 'FontWeight', 'bold');
165 legend('Location', 'best', 'FontSize', 11);
166 xlim([0 10]); ylim([0 120]);
167 saveas(fig, [OUTPUT_DIR 'all_regulators_comparison.png']); hold off;
168
169 fig = figure('Position', [100, 100, 1200, 800]);
170 hold on; grid on;
171 plot(relay_100(:,1), relay_100(:,2)-TARGET, 'k-', 'LineWidth', 2, '
    DisplayName', 'Relay');
172 plot(p_good(:,1), p_good(:,2)-TARGET, 'r-', 'LineWidth', 2, '
    DisplayName', 'P');
173 plot(pd_good(:,1), pd_good(:,2)-TARGET, 'g-', 'LineWidth', 2, '
    DisplayName', 'PD');
174 plot(pi_good(:,1), pi_good(:,2)-TARGET, 'b-', 'LineWidth', 2, '
    DisplayName', 'PI');
175 plot(pid_exp(:,1), pid_exp(:,2)-TARGET, 'm-', 'LineWidth', 2, '
    DisplayName', 'PID');
176 plot(pdi_exp(:,1), pdi_exp(:,2)-TARGET, 'c-', 'LineWidth', 2, '
    DisplayName', 'PDI');
177 yline(0, '--k', 'Zero error', 'LineWidth', 2);
178 xlabel('Time (s)', 'FontSize', 13); ylabel('Error e(t) = \textbackslash
    theta(t) - \textbackslash theta^{\ast} (deg)', 'FontSize', 13);
179 title('Regulation Error Comparison', 'FontSize', 15, 'FontWeight', 'bold');
180 legend('Location', 'best', 'FontSize', 11);
181 ylim([-20 20]);
182 saveas(fig, [OUTPUT_DIR 'error_comparison.png']); hold off;
183
184 fig = figure('Position', [100, 100, 1200, 800]);
185 hold on; grid on;
186 plot(relay_100(:,1), relay_100(:,3), 'k-', 'LineWidth', 2, 'DisplayName', '
    Relay');
187 plot(p_good(:,1), p_good(:,3), 'r-', 'LineWidth', 2, 'DisplayName', 'P
    ');
188 plot(pd_good(:,1), pd_good(:,3), 'g-', 'LineWidth', 2, 'DisplayName', '
    PD');
189 plot(pi_good(:,1), pi_good(:,3), 'b-', 'LineWidth', 2, 'DisplayName', '
    PI');
190 plot(pid_exp(:,1), pid_exp(:,3), 'm-', 'LineWidth', 2, 'DisplayName', '
    PID');
191 plot(pdi_exp(:,1), pdi_exp(:,3), 'c-', 'LineWidth', 2, 'DisplayName', '
    PDI');
192 xlabel('Time (s)', 'FontSize', 13); ylabel('Control Signal U (%)', 'FontSize
    ', 13);
193 title('Control Signals Comparison', 'FontSize', 15, 'FontWeight', 'bold');
194 legend('Location', 'best', 'FontSize', 11);
195 ylim([-150 150]);
196 saveas(fig, [OUTPUT_DIR 'control_signals_comparison.png']); hold off;
197
198 fprintf('All graphs successfully created!\n');
199 fprintf('Saved to: %s\n', OUTPUT_DIR);

```

Листинг 5: generate_all_plots.m